МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:
Босак В.Ю.
студент группы
ПО-13
Проверил:
А.Д.Кулик
13.03.2026

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Ход работы

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
4. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Выполнение: Код программы:

```
from datetime import datetime, date, time
from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey, Float, Date,
Time
from sqlalchemy.orm import sessionmaker, declarative_base, relationship, Session

DATABASE_URL = "sqlite:///transport.db"

engine = create_engine(DATABASE_URL, echo=True)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

# SQLAlchemy модели
class Route(Base):
    __tablename__ = "routes"

    id = Column(Integer, primary_key=True)
    number = Column(String, unique=True, nullable=False)
    start_point = Column(String, nullable=False)
    end_point = Column(String, nullable=False)

    buses = relationship("Bus", back_populates="route")

class Bus(Base):
    __tablename__ = "buses"

    id = Column(Integer, primary_key=True)
    route_id = Column(Integer, ForeignKey("routes.id"))
    model = Column(String, nullable=False)
    year = Column(Integer, nullable=False)
    capacity = Column(Integer, nullable=False)

    route = relationship("Route", back_populates="buses")
    driver = relationship("Driver", back_populates="bus", uselist=False)
    trips = relationship("Trip", back_populates="bus")

class Driver(Base):
    __tablename__ = "drivers"

    id = Column(Integer, primary_key=True)
    bus_id = Column(Integer, ForeignKey("buses.id"), unique=True)
    name = Column(String, nullable=False)
    license_number = Column(String, unique=True, nullable=False)
```

```
    phone = Column(String)

    bus = relationship("Bus", back_populates="driver")
    trips = relationship("Trip", back_populates="driver")

class Stop(Base):
    __tablename__ = "stops"

    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    address = Column(String)
    latitude = Column(Float)
    longitude = Column(Float)

class Trip(Base):
    __tablename__ = "trips"

    id = Column(Integer, primary_key=True)
    bus_id = Column(Integer, ForeignKey("buses.id"))
    route_id = Column(Integer, ForeignKey("routes.id"))
    driver_id = Column(Integer, ForeignKey("drivers.id"))
    trip_date = Column(Date, nullable=False)
    trip_time = Column(Time, nullable=False)

    bus = relationship("Bus", back_populates="trips")
    route = relationship("Route")
    driver = relationship("Driver", back_populates="trips")

Base.metadata.create_all(bind=engine)

# Pydantic модели для ответов
class RouteOut(BaseModel):
    id: int
    number: str
    start_point: str
    end_point: str

class BusOut(BaseModel):
    id: int
    route_id: int
    model: str
    year: int
    capacity: int
```

```
class DriverOut(BaseModel):
    id: int
    bus_id: int
    name: str
    license_number: str
    phone: str

class StopOut(BaseModel):
    id: int
    name: str
    address: str
    latitude: float
    longitude: float

class TripOut(BaseModel):
    id: int
    bus_id: int
    route_id: int
    driver_id: int
    trip_date: date
    trip_time: time

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.post("/routes/")
def create_route(number: str, start_point: str, end_point: str, db: Session =
Depends(get_db)):
    route = Route(number=number, start_point=start_point, end_point=end_point)
    db.add(route)
    db.commit()
    db.refresh(route)
    return RouteOut(id=route.id, number=route.number, start_point=route.start_point,
end_point=route.end_point)

@app.get("/routes/", response_model=list[RouteOut])
def get_routes(db: Session = Depends(get_db)):
    routes = db.query(Route).all()
```

```
    return routes

@app.put("/routes/{route_id}")
def update_route(route_id: int, number: str, start_point: str, end_point: str, db: Session =
Depends(get_db)):
    route = db.query(Route).filter(Route.id == route_id).first()
    if not route:
        raise HTTPException(status_code=404, detail="Route not found")
    route.number = number
    route.start_point = start_point
    route.end_point = end_point
    db.commit()
    return RouteOut(id=route.id, number=route.number, start_point=route.start_point,
end_point=route.end_point)

@app.delete("/routes/{route_id}")
def delete_route(route_id: int, db: Session = Depends(get_db)):
    route = db.query(Route).filter(Route.id == route_id).first()
    if not route:
        raise HTTPException(status_code=404, detail="Route not found")
    db.delete(route)
    db.commit()
    return {"status": "deleted"}

@app.post("/buses/")
def create_bus(route_id: int, model: str, year: int, capacity: int, db: Session =
Depends(get_db)):
    bus = Bus(route_id=route_id, model=model, year=year, capacity=capacity)
    db.add(bus)
    db.commit()
    db.refresh(bus)
    return BusOut(id=bus.id, route_id=bus.route_id, model=bus.model, year=bus.year,
capacity=bus.capacity)

@app.get("/buses/", response_model=list[BusOut])
def get_buses(db: Session = Depends(get_db)):
    return db.query(Bus).all()

@app.put("/buses/{bus_id}")
def update_bus(bus_id: int, route_id: int, model: str, year: int, capacity: int, db: Session =
Depends(get_db)):
    bus = db.query(Bus).filter(Bus.id == bus_id).first()
    if not bus:
        raise HTTPException(status_code=404, detail="Bus not found")
```

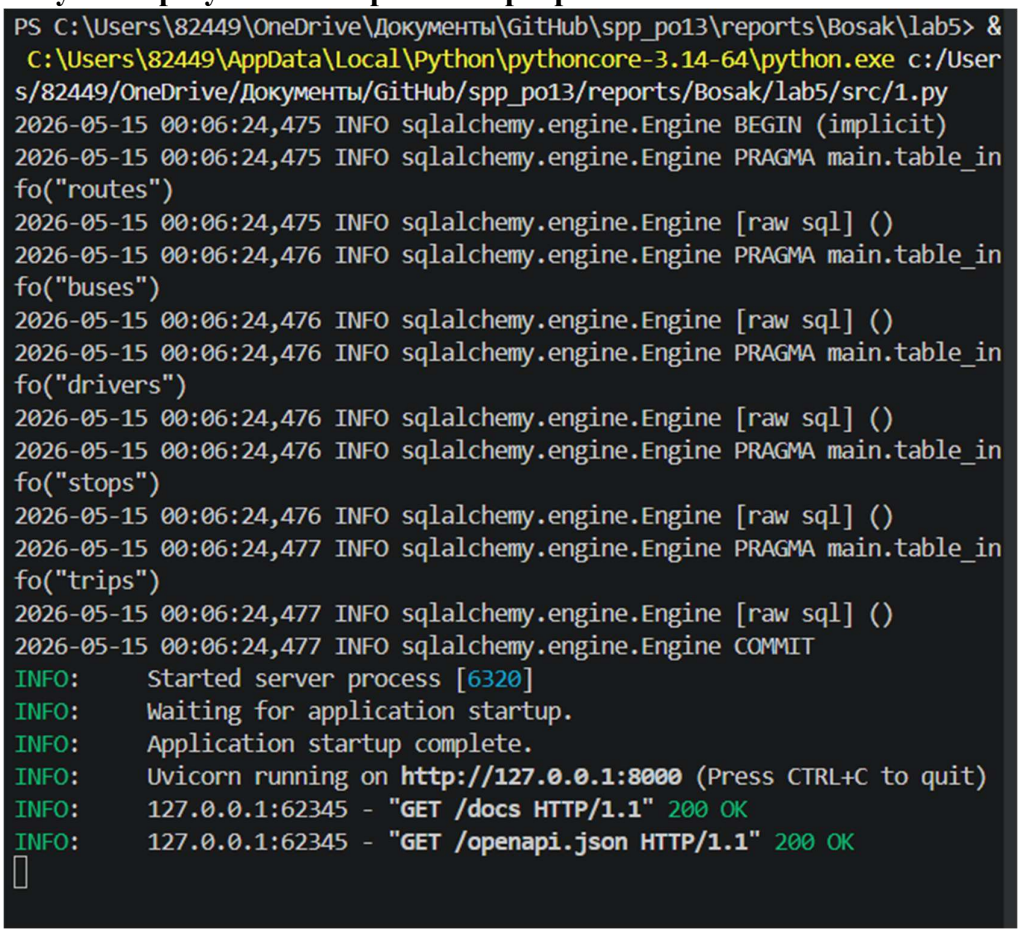
```
    bus.route_id = route_id
    bus.model = model
    bus.year = year
    bus.capacity = capacity
    db.commit()
    return BusOut(id=bus.id, route_id=bus.route_id, model=bus.model, year=bus.year,
capacity=bus.capacity)

@app.delete("/buses/{bus_id}")
def delete_bus(bus_id: int, db: Session = Depends(get_db)):
    bus = db.query(Bus).filter(Bus.id == bus_id).first()
    if not bus:
        raise HTTPException(status_code=404, detail="Bus not found")
    db.delete(bus)
    db.commit()
    return {"status": "deleted"}

@app.post("/drivers/")
def create_driver(bus_id: int, name: str, license_number: str, phone: str, db: Session =
Depends(get_db)):
    driver = Driver(bus_id=bus_id, name=name, license_number=license_number,
phone=phone)
    db.add(driver)
    db.commit()
    db.refresh(driver)
    return DriverOut(id=driver.id, bus_id=driver.bus_id, name=driver.name,
license_number=driver.license_number, phone=driver.phone)

@app.get("/drivers/", response_model=list[DriverOut])
def get_drivers(db: Session = Depends(get_db)):
    return db.query(Driver).all()

@app.put("/drivers/{driver_id}")
def update_driver(driver_id: int, bus_id: int, name: str, license_number: str, phone: str, db:
Session = Depends(get_db)):
    driver = db.query(Driver).filter(Driver.id == driver_id).first()
    if not driver:
        raise HTTPException(status_code=404, detail="Driver not found")
    driver.bus_id = bus_id
    driver.name = name
    driver.license_number = license_number
    driver.phone = phone
    db.commit()
```

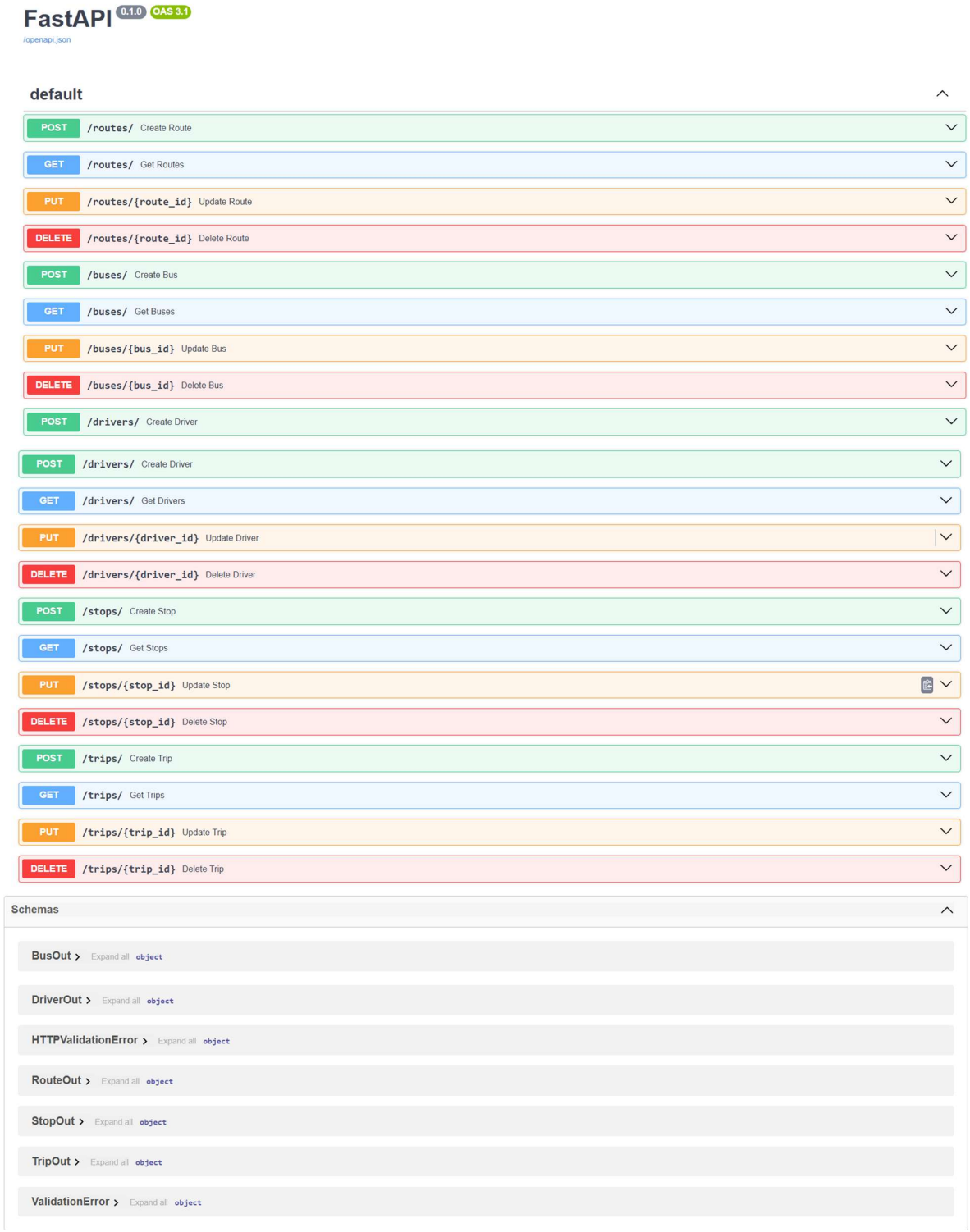
```
    return DriverOut(id=driver.id, bus_id=driver.bus_id, name=driver.name,
license_number=driver.license_number, phone=driver.phone)

@app.delete("/drivers/{driver_id}")
def delete_driver(driver_id: int, db: Session = Depends(get_db)):
    driver = db.query(Driver).filter(Driver.id == driver_id).first()
    if not driver:
        raise HTTPException(status_code=404, detail="Driver not found")
    db.delete(driver)
    db.commit()
    return {"status": "deleted"}

@app.post("/stops/")
def create_stop(name: str, address: str, latitude: float, longitude: float, db: Session =
Depends(get_db)):
    stop = Stop(name=name, address=address, latitude=latitude, longitude=longitude)
    db.add(stop)
    db.commit()
    db.refresh(stop)
    return StopOut(id=stop.id, name=stop.name, address=stop.address,
latitude=stop.latitude, longitude=stop.longitude)

@app.get("/stops/", response_model=list[StopOut])
def get_stops(db: Session = Depends(get_db)):
    return db.query(Stop).all()

@app.put("/stops/{stop_id}")
def update_stop(stop_id: int, name: str, address: str, latitude: float, longitude: float, db:
Session = Depends(get_db)):
    stop = db.query(Stop).filter(Stop.id == stop_id).first()
    if not stop:
        raise HTTPException(status_code=404, detail="Stop not found")
    stop.name = name
    stop.address = address
    stop.latitude = latitude
    stop.longitude = longitude
    db.commit()
    return StopOut(id=stop.id, name=stop.name, address=stop.address,
latitude=stop.latitude, longitude=stop.longitude)

@app.delete("/stops/{stop_id}")
def delete_stop(stop_id: int, db: Session = Depends(get_db)):
    stop = db.query(Stop).filter(Stop.id == stop_id).first()
    if not stop:
```

```
        raise HTTPException(status_code=404, detail="Stop not found")
    db.delete(stop)
    db.commit()
    return {"status": "deleted"}

@app.post("/trips/")
def create_trip(bus_id: int, route_id: int, driver_id: int, trip_date: date, trip_time: time, db:
Session = Depends(get_db)):
    trip = Trip(bus_id=bus_id, route_id=route_id, driver_id=driver_id, trip_date=trip_date,
trip_time=trip_time)
    db.add(trip)
    db.commit()
    db.refresh(trip)
    return TripOut(id=trip.id, bus_id=trip.bus_id, route_id=trip.route_id,
driver_id=trip.driver_id, trip_date=trip.trip_date, trip_time=trip.trip_time)

@app.get("/trips/", response_model=list[TripOut])
def get_trips(db: Session = Depends(get_db)):
    return db.query(Trip).all()

@app.put("/trips/{trip_id}")
def update_trip(trip_id: int, bus_id: int, route_id: int, driver_id: int, trip_date: date,
trip_time: time, db: Session = Depends(get_db)):
    trip = db.query(Trip).filter(Trip.id == trip_id).first()
    if not trip:
        raise HTTPException(status_code=404, detail="Trip not found")
    trip.bus_id = bus_id
    trip.route_id = route_id
    trip.driver_id = driver_id
    trip.trip_date = trip_date
    trip.trip_time = trip_time
    db.commit()
    return TripOut(id=trip.id, bus_id=trip.bus_id, route_id=trip.route_id,
driver_id=trip.driver_id, trip_date=trip.trip_date, trip_time=trip.trip_time)

@app.delete("/trips/{trip_id}")
def delete_trip(trip_id: int, db: Session = Depends(get_db)):
    trip = db.query(Trip).filter(Trip.id == trip_id).first()
    if not trip:
        raise HTTPException(status_code=404, detail="Trip not found")
    db.delete(trip)
    db.commit()
    return {"status": "deleted"}
```

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

Рисунки с результатами работы программы





Вывод: приобрёл практические навыки разработки API и баз данных